

Sorbonne Université
MU4IN019 – Nuage et Réseaux Virtuels

TME 1

Virtualisation complète et réseaux virtuels

Auteurs :

HUANG Loïc & HARRAOUI Ines

Année universitaire 2025–2026

Table des matières

1	I. Virtualisation complète	2
2	II. Création de VM QEMU/KVM avec Virtual Manager	3
3	III. Connectivité de la VM et réseau virtuel	4
4	IV. Réseaux de machines virtuelles	6

I. Virtualisation complète

1. *Est-ce qu'on a activé le support de la technologie de virtualisation pour les PCs se basant sur les processeurs x86 ? Justifiez.*

Réponse

Oui, car on est en virtualisation VT-x sous l'architecture **x86_64**.

2. *Combien y a-t-il de processeurs réels (CPU) ?*

Réponse

Il y a en tout **12 processeurs réels (CPU)**, visible grâce à la ligne "*Processeur(s)*" de la commande `lscpu`.

3. *Les drapeaux (flags) des processeurs montrant le support de la virtualisation sont-ils présents ? Lesquels ?*

Réponse

Oui, ils sont présents : on trouve le drapeau **VMX**.

4. *À partir d'un terminal de la VM Debian lancée avec VBox, répondez aux questions suivantes :*

- a. *Quel est le type de virtualisation utilisé par la VM Debian ?*

Réponse

Le type de virtualisation est une **virtualisation complète**. Le logiciel de virtualisation est **KVM**.

- b. *Combien de vCPU y a-t-il dans la VM ?*

Réponse

Il y a **6 vCPUs** dans la VM.

- c. *Est-ce que les vCPUs supportent à leur tour la technologie de virtualisation (virtualisation imbriquée / nested virtualization) ? Cherchez VT-x et/ou vmx.*

Réponse

Non, les vCPUs **ne supportent pas** la virtualisation imbriquée car les flags VT-x et vmx sont absents.

- d. *Comment peut-on distinguer que les vCPUs sont gérés par un hyperviseur (cherchez dans les flags) ?*

Réponse

On trouve le flag `hypervisor` dans la liste des flags du processeur.

e.

Quel est le constructeur/éditeur de l'hyperviseur avec lequel le processeur interagit ? (La réponse commence par la lettre K.)

Réponse

Le constructeur est **KVM** (*Kernel-based Virtual Machine*).

f.

La commande `dmidecode -s system-product-name` ou `dmidecode -s system-manufacturer` vous permet de trouver des informations sur la machine et son fabricant. Que donne-t-elle ?

Réponse

La machine est une **VirtualBox** et son fabricant est **innotekGmbH**.

II. Création de VM QEMU/KVM avec Virtual Manager

1.

Trouvez la ligne dans l'affichage de `pciconf -l` qui correspond à l'élément `<controller type='ide'>` du fichier `freebsd.xml`.

Réponse

```
atapci@pci0:0:1:1:
```

2.

Trouvez la ligne correspondant à l'interface PCI de la carte réseau virtuelle (émulée) de la VM FreeBSD, et sa configuration dans `freebsd.xml`.

Réponse

Interface PCI : `em0@pci0:0:1:1`
 Dans `freebsd.xml` :

```
<address type='pci' domain='0x0000' bus='0x00'
        slot='0x03' function='0x0' />
```

3.

Combien de CPUs FreeBSD a-t-il détectés ? Leurs noms ? Modèle/fabricant ? Taille de la RAM ? Correspond-elle à 2 Go (2048 MiB) ?

Réponse

FreeBSD a détecté **2 CPUs** : `cpu0` et `cpu1`.
 Modèle : **QEMU Virtual CPU version 2.5+** – Fabricant : **AMD**
 Taille RAM : **2048 MB** – correspond bien à 2 Go.

4. Tapez `ifconfig` dans la VM FreeBSD. Quelle est l'adresse MAC de l'interface réseau ? Est-elle identique à celle de Virtual Manager ?

Réponse

Adresse MAC : 52:54:00:a7:c1:f7 – identique à celle indiquée dans Virtual Manager.

5. Depuis la VM FreeBSD, testez `ping` vers l'adresse IP de la carte réseau de la VM Linux hôte. Quel est le résultat ?

Réponse

Le ping **fonctionne** depuis la VM FreeBSD vers la VM Linux hôte.

6. Depuis la VM FreeBSD, testez `ping` vers l'adresse IP de la machine réelle (10.4.0.xx). Quel est le résultat ?

Réponse

Le ping **fonctionne**. La VM FreeBSD peut accéder à Internet **en passant par les autres VMs** qui jouent le rôle d'intermédiaires (routage, NAT, etc.).

III. Connectivité de la VM et réseau virtuel

1. Combien de réseaux virtuels ont été créés ? Pour chaque réseau : nom, périphérique associé, adresse, comment les VMs obtiennent-elles une IP, comment accèdent-elles à Internet ?

Réponse

Un seul réseau virtuel a été créé :

- **Nom** : default
- **Périphérique** : virbr0
- **Adresse réseau** : 192.168.122.0/24
- **Attribution IP** : via **DHCP** (attribution dynamique d'adresses IP aux machines qui se connectent)
- **Accès Internet** : via **NAT** – le NAT traduit les adresses IP privées en adresse IP publique, résolvant le problème d'épuisement des adresses IPv4

2. Dans les paramètres de la VM FreeBSD, à quel réseau est-elle connectée ?

Réponse

La VM FreeBSD est connectée au réseau **“default”**.

3. Quels sont les autres modes ou méthodes permettant de connecter la VM à un réseau ?

Réponse

- Se connecter aux périphériques de l'hôte `enp0s3` : **macvtap**
- Choisir un nom d'un périphérique partagé

4.

À quoi correspond en réalité le réseau virtuel “default” ? C’est quoi exactement le périphérique `virbr0` ? (Commandes : `ip link show type bridge`, `brctl show`)

Réponse

Le réseau virtuel “default” est en réalité un **bridge** (commutateur Ethernet virtuel).

5.

Le bridge `virbr0` n’est pas directement connecté à un lien physique. Expliquez comment les VMs accèdent à Internet. Vérifiez avec `iptables -t nat -L -n -v | grep 192.168`. Reportez les résultats de `ping`, `cat /proc/net/nf_conntrack` et `netstat-nat -nN`.

Réponse

Le bridge `virbr0` est associé à la pile protocolaire de la VM Linux Debian (hôte). Pour acheminer les trames de la VM FreeBSD vers Internet, la solution choisie par défaut par `virt-manager` est le **NAT (MASQUERADE)** via `iptables`.

Fonctionnement du NAT MASQUERADE : remplace l’adresse IP source de tous les paquets sortants par l’adresse de l’interface de sortie. Les numéros de port sont randomisés si nécessaire. Les paquets ICMP sont mappés via leurs identifiants.

- Adresse IP utilisée pour la translation : `192.168.122.98`
- Interface où la translation est effectuée : `enp0s3` (`10.4.0.119`)

Image 1 : `ping 10.4.0.3`

Image 2 : Résultat de la commande `cat /proc/net/nf_conntrack`

Image 3 : Résultat de la commande `netstat-nat -nN`

6.

Comment la VM FreeBSD est-elle connectée au bridge `virbr0` ? (Commandes : `ip link show up | grep master`, `ip link show type tun up`)

Réponse

La VM FreeBSD est connectée au bridge `virbr0` via l’interface `vnet0` de type **TUN/TAP**.

- Pourquoi pas d’adresse IP sur cette interface ?** Elle transmet directement les trames au noyau du système hôte sans passer par des câbles physiques. Elle joue le rôle de connecteur entre le bridge et la VM FreeBSD.
- Type de l’interface : TUN/TAP** (périphérique réseau virtuel du noyau).
- Destination finale des données** : la VM FreeBSD. C’est la **carte réseau émulée par QEMU** qui fait le lien entre l’interface TUN/TAP et la VM.
- Mode de l’interface : TAP** (vérifiable avec `ip tuntap | grep vnet0`).

7. Vérifiez que FreeBSD repose sur DHCP (fichier `/etc/rc.conf`). Répondez aux sous-questions sur le serveur DHCP.

Réponse

- a. Quel système joue le rôle de serveur DHCP ? La VM hôte (Linux Debian).
- b. Nom du processus DHCP : dnsmasq
- c. Fichier de configuration : `default.conf`
Intervalle d'adresses IP : 192.168.122.2 → 192.168.122.254
- d. Adresse du serveur DNS (fichier `/etc/resolv.conf` de FreeBSD) : 192.168.122.1

IV. Réseaux de machines virtuelles

Rappel – Format QCOW2

Le format **QCOW2** (*Qemu Copy-On-Write*) permet de créer, à partir d'une image de base (*backing file*), des images légères de superposition (*overlay*). Seules les **différences** avec l'image de base sont enregistrées. Particulièrement utile dans les environnements Cloud pour créer massivement des ressources.

1. Quel est le nom du périphérique associé au réseau `testres` (nom du bridge) ?

Réponse

Le périphérique associé au réseau `testres` est `virbr1`.

2. Vérifiez les notions de superposition et copy-on-write :

- a. Avec `df -h` dans `freebsd1`, déterminez la taille globale utilisée du disque. Comparez avec `ls -lh` dans l'hôte.

Réponse

- Taille globale utilisée (dans la VM) : **≈ 121 Go**
- Taille de `freebsd` (image de base) : **7,1 Go**
- Taille de `freebsd1.qcow2` (overlay) : **12 Mo**

Le format qcow2 permet donc de réduire drastiquement la taille sur disque pour créer davantage de VMs.

- b. Tapez `qemu-img info freebsd1.qcow2` dans l'hôte et comparez avec les résultats précédents.

Réponse

La taille affichée par `ls -lh` (**12 Mo**) est équivalente à la *disk size* retournée par `qemu-img info freebsd1.qcow2`.

Dans `freebsd1`, créez un fichier de 10 Mo avec `dd if=/dev/zero of=fichier bs=1000 count=10000`. Vérifiez que la taille de `freebsd1.qcow2` a augmenté d'environ 10 Mo.

c.

Réponse

La taille de `freebsd1.qcow2` est passée de **12 Mo** à **23 Mo** : augmentation de 10 Mo confirmée.

3.

Vérifiez que les 3 VMs clonées ont les mêmes propriétés (CPUs, RAM, souris, clavier).

Réponse

Les 3 VMs clonées ont bien les mêmes propriétés : nombre de CPUs, taille de la RAM, souris et clavier.

4.

Quelles sont les adresses IP et MAC de chaque clone ? Vérifiez avec `ping` que les trois clones sont bien connectés entre eux.

Réponse

VM	Adresse MAC	Adresse IP
freebsd1	52:54:00:f0:cb:79	172.27.27.6
freebsd2	52:54:00:c9:47:60	172.27.27.8
freebsd3	52:54:00:06:85:e6	172.27.27.5

Les pings entre les trois clones fonctionnent.

5.

Donnez les noms des interfaces TAP associées aux clones.

Réponse

VM	Interface TAP
freebsd1	vnet1
freebsd2	vnet2
freebsd3	vnet0

6.

Essayez de créer un autre réseau virtuel `res2` avec la même adresse réseau `172.27.27.0/24` que `testres`. Décrivez le résultat.

Réponse

Une erreur est levée :

```
Erreur lors de la creation du reseau virtuel :
internal error : Network is already in use
by interface virbr1.
```

Il est donc impossible d'avoir deux réseaux virtuels avec la même adresse réseau.